

EXAMEN PRÁCTICO — UNIDAD 1

Generalidades de los Sistemas Distribuidos

Datos generales

Asignatura	Aplicaciones Distribuidas (ISR-701)
Unidad	1 — Generalidades de los Sistemas Distribuidos
Tipo de evaluación	Examen práctico de unidad — Individual
Modalidad	Laboratorio con defensa técnica oral
Duración	2 horas de laboratorio
Período académico	Regular 2025-2026 SPA
Código del instrumento	EV-PR-U1 (ajustable según planificación)
Docente	_____

Estudiante	Cando Moreno Robinson Rodrigo
Fecha	02/06/2026
URL del repositorio	https://github.com/Robimson/APP_distribuidas.git
Calificación (sobre 10)	

1. Competencia y resultado de aprendizaje evaluados

Este examen valora la capacidad del estudiante para analizar las características y principios de los sistemas distribuidos, evaluando sus ventajas, limitaciones y retos desde una perspectiva crítica, y para implementar de forma funcional los mecanismos fundamentales de la unidad: comunicación entre procesos, sincronización mediante relojes lógicos, tolerancia a fallos, coordinación y seguridad básica.

2. Planteamiento del problema

Una empresa de logística opera un servicio de registro de operaciones llamado RegistroDistribuido. El servicio se compone de tres nodos réplica (N1, N2, N3) que reciben “operaciones” (por ejemplo, registros de movimientos) provenientes de uno o varios clientes. Por razones de disponibilidad, los nodos deben coordinarse entre sí, mantener un orden coherente de los eventos que registran y seguir funcionando aunque alguno de ellos deje de responder.

Su tarea es construir un prototipo funcional de este servicio que demuestre, sobre un caso pequeño y ejecutable, los principios estudiados en la Unidad 1. No se evalúa la escala ni la persistencia en disco, sino la correcta aplicación de los mecanismos distribuidos. El sistema puede ejecutarse como tres procesos en la misma máquina (distintos puertos) o como tres contenedores.

3. Requerimientos a implementar

Parte A — Comunicación entre procesos (peso orientativo 20%)

- Implemente la comunicación cliente–nodo (y nodo–nodo) usando sockets TCP o gRPC.

- Delimite correctamente los mensajes (longitud previa, separador o framing) de modo que dos operaciones enviadas seguidas no se solapen ni se trunquen en el receptor.
- Un cliente debe poder enviar una operación y recibir la confirmación del nodo.

Parte B — Relojes lógicos de Lamport (peso orientativo 20%)

- Cada nodo mantiene un reloj lógico de Lamport e implementa correctamente sus reglas: incremento ante un evento local y, al recibir un mensaje, fijar el reloj en $\max(\text{local}, \text{recibido}) + 1$.
- El sistema debe generar un registro (log) de eventos con un orden total consistente, usando el identificador de nodo como criterio de desempate ante marcas iguales.

Parte C — Tolerancia a fallos (peso orientativo 15%)

- Los nodos intercambian latidos (heartbeats) periódicos.
- Si un nodo deja de responder, los demás lo marcan como caído dentro de un tiempo acotado, registran el evento y el sistema continúa operando con los nodos restantes (gestión de fallo parcial).

Parte D — Coordinación (peso orientativo 10%)

- Implemente la elección de coordinador con el algoritmo Bully: el nodo activo de mayor identificador actúa como coordinador.
- Ante la caída del coordinador, el sistema debe disparar y completar una nueva elección sin intervención manual.

Parte E — Seguridad (peso orientativo 10%)

- El cliente debe autenticarse mediante un token antes de que el nodo acepte sus operaciones; las solicitudes sin token válido se rechazan.

4. Componente de análisis y defensa técnica

Al finalizar la implementación, el estudiante registrará sus respuestas en un archivo ANALISIS.md y las defenderá oralmente ante el docente. Las preguntas conectan el prototipo con la teoría de la unidad:

1. Ante una interrupción de comunicación entre dos nodos, ¿qué propiedad del teorema CAP privilegia su implementación y por qué? Justifíquelo con el comportamiento observado en su prototipo.
2. ¿Qué falacias de la computación distribuida tuvo que considerar al delimitar los mensajes y al definir los tiempos de espera de los latidos?
3. ¿Qué tipos de transparencia (ubicación, acceso, fallos, replicación) ofrece o no ofrece su solución? Argumente cada caso.
4. Proponga un acuerdo de nivel de servicio (SLA) de disponibilidad para este sistema y calcule el tiempo de inactividad anual admisible que implicaría.
5. Si reemplazara el algoritmo Bully por un consenso tipo Raft, ¿qué ganaría y qué costo introduciría?

5. Entregables y condiciones

- **Repositorio:** Repositorio Git con el código fuente y un README que indique cómo compilar y ejecutar el prototipo.
- **Análisis:** Archivo ANALISIS.md con las respuestas del componente de análisis.
- **Demostración:** Demostración en vivo: levantar los tres nodos, enviar operaciones, mostrar el orden total de eventos y provocar la caída de un nodo (incluido el coordinador).

- **Condición:** El prototipo debe compilar y ejecutarse siguiendo las instrucciones del README. Un proyecto que no ejecuta no puede acreditar los criterios funcionales.

6. Restricciones técnicas

- Lenguaje y stack: Java 21 + Spring Boot 3.x, o Python 3.11+. Comunicación mediante sockets TCP o gRPC.
- Ejecución local (varios puertos) o en contenedores Docker; IDE de referencia: IntelliJ IDEA.
- El uso de asistentes de IA y de material de consulta se rige por la política indicada por el docente para esta evaluación. En la defensa, el estudiante debe poder explicar y justificar cada parte de su código sin leerlo textualmente.

7. Rúbrica de evaluación

Cada criterio se valora en cuatro niveles: Excelente (4), Bueno (3), Regular (2) y Deficiente (1). Un criterio no entregado, con plagio verificado o que no corresponde recibe 0. La calificación final se obtiene como la suma ponderada de los niveles alcanzados, según la fórmula:

$$\text{Nota (sobre 10)} = [\sum (\text{peso}_i \times \text{nivel}_i) \div 4] \times 10$$

Donde los pesos se expresan como fracción (sumando 1,0) y el nivel es el valor 1 a 4 alcanzado en cada criterio. Si todos los criterios alcanzan el nivel Excelente, la nota es 10

A continuación, se presentará la documentación del examen práctico realizado en clases.

Paquete: bully - Clase: BullyService

```

public class BullyService {

    private final ServerNode nodo;
    private final Map<Integer, Long> lastHeartbeat;

    public BullyService(ServerNode nodo, Map<Integer, Long>
lastHeartbeat) {
        this.nodo = nodo;
        this.lastHeartbeat = lastHeartbeat;
    }

    public void startElection() {

        System.out.println("[N" + nodo.getId() + "] INICIANDO
ELECCION");

        boolean higherExists = false;

        for (int id : lastHeartbeat.keySet()) {

            if (id > nodo.getId()) {
                nodo.send(id, nodo.getId() + "|ELECTION|");
                higherExists = true;
            }
        }

        if (!higherExists) {
            nodo.becomeCoordinator();
        }
    }
}

```

Esta clase es la encargada de que los nodos elijan un líder de forma autónoma cuando detectan que el coordinador actual falló. Para funcionar, necesita conocer quiénes están vivos, por lo que recibe una referencia del nodo local y el mapa de los demás. Cuando se activa el método startElection, el nodo inicia una competencia jerárquica recorre la lista de nodos y envía un mensaje de elección solo a aquellos que tienen un ID más alto que el suyo. Si al final del recorrido se da cuenta de que no hay nadie superior activo, asume el mando llamando a becomeCoordinator. Básicamente, es el mecanismo que evita que el sistema se quede sin jefe permitiendo que los servidores se pongan de acuerdo entre ellos basándose en quién tiene el número de identificación más grande.

Paquete: lamport - Clase: LamportClock

```

package org.uteq.practicaexa.lamport;

public class LamportClock {

    private int time = 0;

    public int tick() {
        return ++time;
    }

    public int update(int received) {
        time = Math.max(time, received) + 1;
    }
}

```

```
        return time;
    }

    public int getTime() {
        return time;
    }
}
```

Esta clase implementa un reloj lógico para solucionar el problema de la falta de sincronización horaria global en sistemas distribuidos, permitiendo que los eventos tengan un orden cronológico coherente mediante números enteros. El método tick se encarga de incrementar el contador cada vez que ocurre un evento local en el nodo, mientras que el método update aplica la regla fundamental de Lamport al recibir un mensaje externo, el nodo compara su tiempo local con el que viene en el mensaje, se queda con el valor más alto y le suma uno. Gracias a esta lógica, se garantiza la causalidad de los eventos, asegurando que cualquier respuesta siempre tenga una marca de tiempo mayor a la del mensaje original y que todos los nodos puedan ordenar las operaciones de forma idéntica.

Paquete: model - Clase: Message

```
public class Message {

    public String type;
    public int sender;
    public int lamport;
    public String data;
    public String token;

    public Message() {}

    public Message(String type, int sender, int lamport, String data,
String token) {
        this.type = type;
        this.sender = sender;
        this.lamport = lamport;
        this.data = data;
        this.token = token;
    }
}
```

Esta clase actúa como el contenedor estándar de información para todos los datos que viajan a través de la red, asegurando que todos los nodos hablen el mismo idioma. Define atributos críticos como el tipo de mensaje type, el ID del emisor sender, la marca de tiempo lógica lamport para el ordenamiento de eventos, y el contenido de la operación data. Además, incluye un campo de seguridad token, diseñado para que el sistema pueda validar la autenticidad del cliente antes de procesar cualquier solicitud. En esencia, es la estructura de datos que permite que la comunicación sea clara, ordenada y segura entre los diferentes componentes del sistema distribuido.

Paquete: node - Clase: MainNode

```
public class MainNode {

    public static void main(String[] args) {

        int id = Integer.parseInt(args[0]);

        ServerNode nodo;

        switch (id) {
            case 1 -> nodo = new ServerNode(1, 9001);
            case 2 -> nodo = new ServerNode(2, 9002);
            case 3 -> nodo = new ServerNode(3, 9003);
            default -> throw new IllegalArgumentException("Nodo
inválido");
        }

        nodo.start();

    }

}
```

Esta clase funciona como el punto de entrada principal del sistema, encargándose de arrancar cada réplica de manera individual a través de parámetros externos. Utiliza lo que es un argumento de consola para identificar qué nodo se desea iniciar y, mediante un bloque switch, le asigna automáticamente su identificador y puerto de red correspondiente 9001, 9002 o 9003. Una vez definida la configuración básica, instancia el servidor y lo pone en marcha llamando al método start. Su rol es importante para la flexibilidad del proyecto, ya que permite desplegar múltiples procesos del servidor en la misma máquina física simplemente cambiando el parámetro de inicio, simulando así un entorno distribuido real de tres nodos independientes.

Paquete: node - Clase: NetworkClient

```
public class NetworkClient {

    public static void send(String host, int port, String message) {

        try {

            Socket socket = new Socket(host, port);
            PrintWriter out = new
PrintWriter(socket.getOutputStream(), true)
        } {
            out.println(message);

        } catch (Exception e) {
            System.out.println("Error enviando a nodo: " +
e.getMessage());
        }

    }

}
```

Esta clase es la herramienta encargada de gestionar el envío de datos hacia el exterior a través de la red, funcionando como el cliente en la arquitectura socket TCP. Proporciona un método estático que abre una conexión directa hacia una dirección IP y puerto

específicos, transmite la cadena de texto del mensaje y asegura que el canal se cierre correctamente una vez completada la operación. Es el componente fundamental que permite que un nodo pueda comunicarse con sus pares o que un cliente externo envíe operaciones, abstrayendo toda la complejidad técnica de la conexión en un solo método reutilizable por el sistema.

Paquete: org.uteq.practicaexa - Clase: PracticaexaApplication

```
@SpringBootApplication
public class PracticaexaApplication {

    public static void main(String[] args) {
        SpringApplication.run(PracticaexaApplication.class, args);
    }

}
```

Esta clase representa el punto de entrada estándar para el framework Spring Boot, encargándose de inicializar el contexto de la aplicación y gestionar la configuración automática del proyecto. Mediante la anotación `@SpringBootApplication`, el sistema activa el escaneo de componentes y la gestión de dependencias, mientras que el método `main` dispara el arranque del motor del framework.

Paquete: org.uteq.practicaexa - Clase: ServerNodo

```
public class ServerNodo {

    private final int id;
    private final int puerto;

    private final LamportClock clock = new LamportClock();
    private final ExecutorService pool =
        Executors.newFixedThreadPool(10);

    private final Map<Integer, Long> lastHeartbeat = new
        ConcurrentHashMap<>();
    private final long TIMEOUT = 5000;

    public ServerNodo(int id, int puerto) {
        this.id = id;
        this.puerto = puerto;

        lastHeartbeat.put(1, System.currentTimeMillis());
        lastHeartbeat.put(2, System.currentTimeMillis());
        lastHeartbeat.put(3, System.currentTimeMillis());
    }

    public void start() {

        System.out.println("Nodo N" + id + " iniciando en puerto " +
            puerto);

        startHeartbeatSender();
    }

}
```

```

startFailureDetector();

try (ServerSocket serverSocket = new ServerSocket(puerto)) {
    while (true) {
        Socket socket = serverSocket.accept();
        pool.execute(() -> handleClient(socket));
    }
} catch (IOException e) {
    System.out.println("Error nodo " + id + ": " +
e.getMessage());
}
}

```

En esta primera sección define la identidad y las herramientas básicas del nodo. Se utiliza un ConcurrentHashMap para rastrear de forma segura la salud de los demás nodos y un ExecutorService que permite atender hasta 10 clientes simultáneamente sin que el servidor se bloquee. El método start es el motor principal activa las tareas de vigilancia en segundo plano y abre un ServerSocket que se queda escuchando conexiones entrantes de forma permanente, delegando cada nueva conexión a un hilo independiente para mantener la alta disponibilidad del servicio.

Clase: ServerNodo parte 2 comunicación y latidos

```

public void sendToNode(int targetId, String message) {

    String host = "localhost";

    int port = switch (targetId) {
        case 1 -> 9001;
        case 2 -> 9002;
        case 3 -> 9003;
        default -> throw new IllegalArgumentException();
    };

    NetworkClient.send(host, port, message);
}

private void startHeartbeatSender() {

    Thread t = new Thread(() -> {
        while (true) {
            try {
                Thread.sleep(2000);

                String msg = id + "|HB|" + System.currentTimeMillis();

                if (id != 1) sendToNode(1, msg);
                if (id != 2) sendToNode(2, msg);
                if (id != 3) sendToNode(3, msg);

            } catch (InterruptedException e) {
                break;
            }
        }
    });
}

```



```
t.setDaemon(true);
t.start();
}
```

En esta sección se gestiona la salida de información y la vigilancia de la red. El método `sendToNode` actúa como un enrutador interno que traduce el ID de un nodo a su dirección física puerto, utilizando el `NetworkClient` para realizar la entrega. Por otro lado, `startHeartbeatSender` ejecuta un hilo en segundo plano que, cada 2 segundos, envía una señal de vida a todos los demás nodos réplica. Este flujo es fundamental ya que sirve para notificar a los compañeros que el nodo local sigue operativo y evitar que lo marquen erróneamente como caído.

Clase: ServerNodo parte 3 detector de fallos

```
private void startFailureDetector() {

    Thread t = new Thread(() -> {
        while (true) {
            try {
                Thread.sleep(3000);

                long now = System.currentTimeMillis();

                for (int nodeId : lastHeartbeat.keySet()) {

                    if (nodeId == id) continue;

                    long last = lastHeartbeat.get(nodeId);

                    if (now - last > TIMEOUT) {
                        System.out.println("[N" + id + "] ✖ Nodo " +
nodeId + " CAÍDO");
                    }
                }

            } catch (InterruptedException e) {
                break;
            }
        }
    });

    t.setDaemon(true);
    t.start();
}
```

Esta parte del método implementa la lógica de Tolerancia a Fallos mediante un sistema de vigilancia activa. Se trata de un hilo que despierta cada 3 segundos para revisar el historial de latidos de los demás nodos. Si el tiempo transcurrido desde el último mensaje recibido supera el límite de 5 segundos `TIMEOUT`, el sistema lo clasifica automáticamente como `CAÍDO`. Esta capacidad es crucial para la gestión de fallos parciales, permitiendo que el sistema distribuido tome conciencia de su estado actual y pueda reaccionar ante la desconexión o muerte de uno de sus integrantes.

Clase: ServerNodo parte 4: procesamiento de mensajes

```
private void handleClient(Socket socket) {

    try (
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));

        PrintWriter out = new PrintWriter(
            socket.getOutputStream(), true)
    ) {

        String line;

        while ((line = in.readLine()) != null) {

            String[] parts = line.split("\\|");

            int sender = Integer.parseInt(parts[0]);
            String type = parts[1];
            String data = parts[2];

            if (type.equals("HB")) {
                lastHeartbeat.put(sender, System.currentTimeMillis());
                continue;
            }

            int receivedLamport = Integer.parseInt(type);

            int newTime = clock.update(receivedLamport);

            System.out.println("[N" + id + "] desde N" + sender +
                " | " + data +
                " | Lamport=" + newTime);

            out.println("ACK N" + id + " lamport=" + newTime);
        }

    } catch (Exception e) {
        System.out.println("Nodo " + id + " cliente desconectado");
    }
}
```

Este método es el receptor central de información y el encargado de aplicar las reglas de sincronización. Primero descompone las cadenas de texto recibidas usando el separador |. Si detecta un mensaje de tipo HB Heartbeat, actualiza el registro de vida del emisor. Si es un mensaje de datos, realiza la actualización del Reloj de Lamport comparando el tiempo recibido con el local, lo que garantiza el orden causal de los eventos. Finalmente, el nodo emite una confirmación ACK incluyendo su nueva marca de tiempo lógica.

Componente de análisis y defensa técnica

1. Ante una interrupción de comunicación entre dos nodos, ¿qué propiedad del teorema CAP privilegia su implementación y por qué? Justifíquelo con el comportamiento observado en su prototipo.

En si el diseño del prototipo privilegia las propiedades de Disponibilidad (A) y Tolerancia a Particiones (P) del teorema CAP, ya que la arquitectura permite que el sistema continúe operando de forma independiente incluso cuando un nodo queda aislado por una falla de red. Al analizar la lógica del detector de fallos basado en latidos, se evidencia que el sistema prefiere mantener el servicio activo para los clientes en lugar de bloquearse esperando una sincronización perfecta con el nodo ausente, aceptando así una consistencia eventual para garantizar que el registro distribuido siga recibiendo operaciones a pesar de las fallas de comunicación.

2. ¿Qué falacias de la computación distribuida tuvo que considerar al delimitar los mensajes y al definir los tiempos de espera de los latidos?

Durante el desarrollo se encontraron falacias críticas como que la red es confiable, la latencia es cero y el ancho de banda es infinito, y en la definición de un tiempo de espera de cinco segundos para absorber retrasos naturales en la transmisión de datos. Asimismo, el uso de una delimitación de mensajes ligera basada en caracteres especiales permite que los paquetes sean eficientes, reconociendo explícitamente que el transporte de información tiene un costo y que el sistema debe ser capaz de reaccionar ante la desaparición de sus componentes sin asumir una conectividad perfecta.

3. ¿Qué tipos de transparencia (ubicación, acceso, fallos, replicación) ofrece o no ofrece su solución? Argumente cada caso.

En cuanto a las transparencias, el sistema ofrece transparencia de acceso al utilizar un protocolo de comunicación por sockets uniforme para todos los componentes y transparencia de fallos al detectar y reportar automáticamente la caída de nodos réplica sin que el sistema colapse por completo. Por otro lado, no se ofrece transparencia de ubicación debido a que los puertos y direcciones están definidos estáticamente en el código fuente, ni una transparencia de replicación total, ya que el cliente interactúa de forma directa con instancias específicas en lugar de ver al conjunto de nodos como una única entidad lógica.

4. Proponga un acuerdo de nivel de servicio (SLA) de disponibilidad para este sistema y calcule el tiempo de inactividad anual admisible que implicaría.

Se propone un acuerdo de nivel de servicio SLA con una disponibilidad del 99.9%, lo que representa un estándar equilibrado para una arquitectura de réplicas que busca tolerancia a fallos. Al realizar el cálculo sobre los 525,600 minutos que conforman un año, esto implicaría un tiempo de inactividad anual admisible de 525.6 minutos, lo que equivale aproximadamente a 8.76 horas al año, cubriendo tanto las ventanas de mantenimiento programadas como los periodos de transición necesarios para que el algoritmo de elección de líder recupere el control tras una caída.

5. Si reemplazara el algoritmo Bully por un consenso tipo Raft, ¿qué ganaría y qué costo introduciría?

Si se reemplazara el algoritmo Bully por un modelo de consenso tipo Raft, el sistema ganaría una consistencia fuerte y una mayor robustez frente a problemas de coordinación complejos, asegurando que todos los nodos tengan exactamente la misma copia de los datos antes de confirmar una transacción. Sin embargo, esto introduciría un costo elevado en términos de complejidad técnica y latencia de red, ya que Raft requiere múltiples rondas de votación y confirmación para cada mensaje enviado, lo que aumentaría la sobrecarga de tráfico y el tiempo de respuesta en comparación con la simplicidad y rapidez que ofrece la lógica actual del algoritmo de Bully.

Evidencia de la prueba realizada

PREGUNTA

1

Finalizado

1.00 / 1.00

Una aplicación en Windows escrita en C# y otra en Linux escrita en Java deben intercambiar mensajes sin que ninguna conozca el sistema operativo ni el lenguaje de la otra. ¿Qué propiedad del middleware lo hace posible y cuál es su costo principal?

Seleccione una alternativa:

- ☒ a. Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización. ✓
- ☐ b. Garantiza que el sistema nunca fallará; el costo es el almacenamiento, que en nuestros días eso no es una limitación.
- ☐ c. Elimina por completo la latencia de red; el costo es el ancho de banda que se cree que es ilimitado por eso no se toma en cuenta su afectación.
- ☐ d. Convierte el sistema distribuido en uno centralizado; el costo es la escalabilidad, dejando al sistema sin las posibilidades de crecer.

La respuesta correcta es: Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización.

PREGUNTA

2

Finalizado

0.00 / 1.00

Un sistema cifra todos los datos en tránsito con TLS, pero almacena las contraseñas en texto plano en la base de datos. Un atacante que compromete la base de datos obtiene todas las contraseñas. ¿Qué principio de seguridad se violó?

Seleccione una alternativa:

- ☐ a. La seguridad en reposo; el cifrado en tránsito no protege los datos almacenados.
- ☐ b. La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados.
- ☒ c. La integridad de los datos en tránsito ✗
- ☐ d. El no repudio o denegación de servicios (DOS), ya que el atacante cambió las contraseñas actuales.

La respuesta correcta es: La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados.

PREGUNTA

3

Finalizado

1.00 / 1.00

Un equipo intenta mejorar el rendimiento de una aplicación que envía miles de mensajes muy pequeños entre dos nodos. Duplica el ancho de banda del enlace, pero el rendimiento prácticamente no cambia. ¿Qué falacia de la computación distribuida explica este resultado?

Seleccione una alternativa:

- ☐ a. "El ancho de banda es infinito".
- ☒ b. "La latencia es cero". ✓
- ☐ c. "La red es confiable"
- ☐ d. "El transporte es gratuito".

La respuesta correcta es: "La latencia es cero".

PREGUNTA

4

Finalizado

1.00 / 1.00

Relacione el escenario y tipo de transparencia evidenciada:

Relacione la opción correcta:

Una página sigue respondiendo con normalidad aunque un servidor del clúster se cayó, sin que el usuario lo note.

Transparencia De Fallos ✓

Cien personas editan el mismo documento a la vez y el sistema preserva el orden correcto de los cambios sin que ellas lo perciban.

Transparencia De Concurrencia ✓

Un usuario abre un archivo sin enterarse de que físicamente reside en un servidor de otra ciudad.

Transparencia De Ubicación ✓

Un objeto se guarda y se recupera de una base de datos sin que el programador administre su almacenamiento en disco.

Transparencia De Persistencia ✓

La respuesta correcta es: Un usuario abre un archivo sin enterarse de que físicamente reside en un servidor de otra ciudad. → Transparencia de ubicación , Un objeto se guarda y se recupera de una base de datos sin que el programador administre su almacenamiento en disco. → Transparencia de persistencia , Cien personas editan el mismo documento a la vez y el sistema preserva el orden correcto de los cambios sin que ellas lo perciban. → Transparencia de concurrencia , Una página sigue respondiendo con normalidad aunque un servidor del clúster se cayó, sin que el usuario lo note. → Transparencia de fallos

PREGUNTA

5

Finalizado

1.00 / 1.00

Un arquitecto debe decidir entre consistencia fuerte (mediante consenso Raft) o consistencia eventual para un contador de "me gusta" en una red social con millones de operaciones por segundo. ¿Cuál es la decisión más razonable y por qué?

Seleccione una alternativa:

- ☐ a. Consistencia fuerte, porque el consenso Raft no introduce latencia adicional, siendo indiferente, porque ambos modelos rinden igual bajo carga elevada
- ☐ b. Consistencia fuerte, porque un conteo exacto en tiempo real es crítico para la seguridad del sistema.
- ☐ c. Consistencia general, porque el dominio tolera imprecisiones generales y conviene priorizar disponibilidad y rendimiento bajo niveles de carga promedio, siendo aceptable que el valor converja después.
- ☒ d. Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después. ✓

La respuesta correcta es: Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después.

PREGUNTA

6

Finalizado

1.00 / 1.00

Un servicio de distribución de archivos observa que, mientras más usuarios se conectan, más capacidad total (ancho de banda y almacenamiento) tiene disponible el sistema. ¿Qué modelo arquitectónico explica mejor esta propiedad?

Seleccione una alternativa:

- ☐ a. Modelo híbrido, porque depende siempre de un servidor central para todas las operaciones.
- ☒ b. Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos. ✓
- ☐ c. Cliente-servidor puro, porque el servidor central crece proporcionalmente con la demanda.
- ☐ d. Cliente-servidor de tres capas, porque separa la lógica de negocio.

La respuesta correcta es: Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.

PREGUNTA

7

Finalizado

1.00 / 1.00

Un desarrollador usa sockets TCP para enviar por separado los mensajes "HOLA" y "MUNDO", pero en el receptor a veces llegan juntos como "HOLAMUNDO" en una sola lectura. ¿Cuál es la explicación correcta?

Seleccione una alternativa:

- ☐ a. El sistema cambió automáticamente a UDP.
- ☒ b. TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente. ✓
- ☐ c. Es un error físico de la tarjeta de red
- ☐ d. TCP perdió y luego retransmitió un mensaje.

La respuesta correcta es: TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente

PREGUNTA

8

Finalizado

0.00 / 1.00

Sockets TCP

Relacione la opción correcta:

Sockets UDP

Invocación remota de métodos (RMI)

Middleware orientado a mensajes (colas)

Comunicación Asíncrona Y Desacoplada, Donde Emisor ✗

Invocación De Métodos Sobre Objetos Remotos Con Paso ✗

Flujo De Bytes Fiable Y Ordenado Que No Preserva Orden ✗

La respuesta correcta es: Middleware orientado a mensajes (colas) → Invocación de métodos sobre objetos remotos con paso de objetos serializados mediante stubs. , Invocación remota de métodos (RMI) → Datagramas sin garantía de entrega ni orden, a cambio de menor latencia. , Sockets UDP → Flujo de bytes fiable y ordenado que no preserva los límites entre mensajes.

PREGUNTA

9

Finalizado

0.00 / 1.00

Un sistema replicado requiere un quórum de mayoría (más de la mitad de los nodos) para confirmar una operación y mantener la consistencia. Con 5 réplicas, ¿cuántos fallos por caída simultáneos puede tolerar sin perder la capacidad de formar quórum?

Seleccione una alternativa:

- ☐ a. 0, porque cualquier caída rompe el quórum.
- ☐ b. 4, porque basta con que una réplica siga viva.
- ☐ c. 2, porque el quórum de mayoría es 3 y se necesitan al menos 3 réplicas vivas
- ☒ d. 3, porque con 2 réplicas vivas todavía hay mayoría. ❌

La respuesta correcta es: 2, porque el quórum de mayoría es 3 y se necesitan al menos 3 réplicas vivas

PREGUNTA

10

Finalizado

0.00 / 1.00

Relacione: Situación de seguridad y propiedad que se protege

Relacione la opción correcta:

El receptor recalcula un hash del mensaje y lo compara con el recibido para detectar si fue alterado en tránsito.

Tolerancia A Fallos ❌

Cada orden de pago se firma digitalmente para que el emisor no pueda negar después haberla emitido.

Autorización ❌

Antes de conceder el acceso, el sistema valida que el usuario es quien dice ser mediante un segundo factor.

Confidencialidad ❌

Un usuario ya autenticado intenta abrir un módulo administrativo y el sistema se lo deniega según su rol.

Autenticación ❌

La respuesta correcta es: Cada orden de pago se firma digitalmente para que el emisor no pueda negar después haberla emitido. → No repudio , Un usuario ya autenticado intenta abrir un módulo administrativo y el sistema se lo deniega según su rol. → Autorización , El receptor recalcula un hash del mensaje y lo compara con el recibido para detectar si fue alterado en tránsito. → Integridad , Antes de conceder el acceso, el sistema valida que el usuario es quien dice ser mediante un segundo factor. → Autenticación

PREGUNTA

11

Finalizado

1.00 / 1.00

Se compara un sistema centralizado con uno distribuido bajo la misma carga. ¿Cuál es una desventaja inherente al distribuido que el centralizado no presenta?

Seleccione una alternativa:

- ☐ a. La imposibilidad de escalar horizontalmente.
- ☐ b. La ausencia total de concurrencia.
- ☐ c. La existencia de un único punto de fallo total del sistema.
- ☒ d. La posibilidad de fallo parcial, donde unos nodos fallan y otros siguen operando, generando estados intermedios inconsistentes. ✔️

La respuesta correcta es: La posibilidad de fallo parcial, donde unos nodos fallan y otros siguen operando, generando estados intermedios inconsistentes.

PREGUNTA

12

Finalizado

1.00 / 1.00

¿Cuál enunciado distingue correctamente la "tolerancia a fallos" de la "prevención de fallos"?

Seleccione una alternativa:

- ☒ a. La tolerancia a fallos busca que el sistema siga operando correctamente a pesar de que los fallos ocurran, mientras que la prevención intenta evitar que ocurran. ✔️
- ☐ b. Son términos sinónimos.
- ☐ c. La tolerancia a fallos elimina por completo todos los fallos posibles, mientras que la prevención pone en alerta al administrador sobre posibles fallos.
- ☐ d. La prevención de fallos solo es aplicable al hardware, mientras que la tolerancia a fallos es tanto al software como al hardware.

La respuesta correcta es: La tolerancia a fallos busca que el sistema siga operando correctamente a pesar de que los fallos ocurran, mientras que la prevención intenta evitar que ocurran.

PREGUNTA

13

Finalizado

0.25 / 1.00

Relaciones: Garantía o comportamiento observado y su modelo/concepto

Relacione la opción correcta:

Tras una escritura, algunas lecturas devuelven valores antiguos durante un lapso, pero todas terminan coincidiendo si cesan las escrituras.

"Lee Tus Propias Escrituras" (Read-Your-Writes)❌

Tras escribir un dato, cualquier lectura posterior en cualquier nodo devuelve ese valor de inmediato.

Consistencia Fuerte (Linealizable)✅

Un proceso siempre ve sus propias escrituras en orden, aunque otros procesos las vean con retraso.

Reloj Lógico De Lamport❌

El sistema confirma una operación únicamente cuando más de la mitad de los nodos la han registrado.

Consistencia Eventual❌

La respuesta correcta es: Tras escribir un dato, cualquier lectura posterior en cualquier nodo devuelve ese valor de inmediato. → Consistencia fuerte (linealizable) , Tras una escritura, algunas lecturas devuelven valores antiguos durante un lapso, pero todas terminan coincidiendo si cesan las escrituras. → Consistencia eventual , El sistema confirma una operación únicamente cuando más de la mitad de los nodos la han registrado. → Quórum de mayoría , Un proceso siempre ve sus propias escrituras en orden, aunque otros procesos las vean con retraso. → "Lee tus propias escrituras" (read-your-writes)

PREGUNTA

14

Finalizado

0.75 / 1.00

Relaciones: Protocolo o algoritmo de coordinación y su propósito clave

Relacione la opción correcta:

Relojes de Lamport

Ordena Causalmente Los Eventos Sin Dependier De Un Reloj✅

Commit en dos fases (2PC)

Divide Una Tabla En Subconjuntos Disjuntos De Filas Distribuidas❌

Algoritmo Bully

Elige Como Coordinador Al Nodo Activo De Mayor Identificador✅

Consenso Raft / Paxos

Logra Acuerdo Sobre Un Único Valor Mediante Quórum De Mayoría✅

La respuesta correcta es: Consenso Raft / Paxos → Logra acuerdo sobre un único valor mediante quórum de mayoría, tolerando caídas de nodos. , Commit en dos fases (2PC) → Garantiza que una transacción se confirme en todos los nodos o en ninguno, aunque puede quedar bloqueado si el coordinador cae. , Algoritmo Bully → Elige como coordinador al nodo activo de mayor identificador tras detectar una caída. , Relojes de Lamport → Ordena causalmente los eventos sin depender de un reloj físico global.

PREGUNTA

15

Finalizado

1.00 / 1.00

En los relojes lógicos de Lamport se cumple que si el evento a ocurre antes que b ($a \rightarrow b$), entonces $C(a) < C(b)$. Sin embargo, observar que $C(x) < C(y)$ NO permite concluir que $x \rightarrow y$. ¿Por qué?

Seleccione una alternativa:

☐ a. Porque los relojes de Lamport no son monótonos crecientes.

☒ b. Porque dos eventos concurrentes (sin relación causal) también pueden recibir marcas ordenadas, de modo que el orden total no implica causalidad.✅

☐ c. Porque Lamport exige relojes físicos perfectamente sincronizados, es decir, que deben dar los mismos valores en los mismos tiempos.

☐ d. Porque $C(x) < C(y)$ siempre implica necesariamente $x \rightarrow y$, para la sincronización de los procesos.

La respuesta correcta es: Porque dos eventos concurrentes (sin relación causal) también pueden recibir marcas ordenadas, de modo que el orden total no implica causalidad.

PREGUNTA

16

Finalizado

1.00 / 1.00

¿Cuál afirmación distingue correctamente un sistema distribuido de un sistema paralelo de memoria compartida?

Seleccione una alternativa:

☐ a. El sistema paralelo no puede tener múltiples procesadores.

☒ b. En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes.✅

☐ c. El sistema distribuido no admite fallos parciales.

☐ d. El sistema distribuido siempre procesa más rápido.

La respuesta correcta es: En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes.

PREGUNTA

17

Finalizado

1.00 / 1.00

Un acuerdo de nivel de servicio (SLA) promete una disponibilidad anual del 99,9 %. ¿Aproximadamente cuánto tiempo de inactividad permite al año, y qué implicaría elevarlo al 99,99 %?

Seleccione una alternativa:

- ☒ a. ≈ 8,76 horas al año; al 99,99 % se reduciría a ≈ 52,6 minutos al año. ✓
- ☐ b. ≈ 3,65 días al año; el 99,99 % no cambiaría nada.
- ☐ c. ≈ 88 horas al año; el 99,99 % lo empeoraría.
- ☐ d. ≈ 1 minuto al año; el 99,99 % lo duplicaría.

La respuesta correcta es: ≈ 8,76 horas al año; al 99,99 % se reduciría a ≈ 52,6 minutos al año.

PREGUNTA

18

Finalizado

1.00 / 1.00

Un servicio se traslada de un servidor a otro durante la madrugada, cuando no hay clientes conectados; al reanudar la operación, los clientes lo siguen invocando con el mismo nombre lógico sin percibir cambio alguno. ¿Qué transparencia se evidencia principalmente?

Seleccione una alternativa:

- ☐ a. Transparencia de reubicación.
- ☒ b. Transparencia de migración. ✓
- ☐ c. Transparencia de ubicación.
- ☐ d. Transparencia de acceso.

La respuesta correcta es: Transparencia de migración.

PREGUNTA

19

Finalizado

0.00 / 1.00

En el protocolo de 2PC, ¿cuál es su principal debilidad y por qué se produce?

Seleccione una alternativa:

- ☐ a. Que requiere relojes físicos sincronizados entre los participantes y que no garantiza la atomicidad de la transacción
- ☐ b. Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" pueden quedar bloqueados indefinidamente esperando la decisión final.
- ☒ c. Que es desbloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" quedan libres del bloqueo indefinidamente esperando la decisión final. ✗
- ☐ d. Que es no determinista en su resultado, en vista de que la comunicación entre las 2PC puede romperse.

La respuesta correcta es: Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" pueden quedar bloqueados indefinidamente esperando la decisión final.

PREGUNTA

20

Finalizado

1.00 / 1.00

Dos centros de datos de un banco mantienen copias sincronizadas de las mismas cuentas. Una falla en el enlace de fibra deja a ambos centros sin comunicación entre sí, aunque cada uno sigue funcionando por separado y recibiendo clientes. Ante esta situación, el sistema decide bloquear todas las operaciones de retiro hasta que se restablezca el enlace, prefiriendo que ningún cliente pueda operar antes que la misma cuenta termine reflejando saldos distintos en cada centro. Según el teorema CAP, ¿qué par de propiedades prioriza esta decisión?

Seleccione una alternativa:

- ☐ a. Disponibilidad y tolerancia a particiones (AP).
- ☐ b. Las tres propiedades simultáneamente
- ☐ c. Consistencia y disponibilidad (CA)
- ☒ d. Consistencia y tolerancia a particiones (CP) ✓

La respuesta correcta es: Consistencia y tolerancia a particiones (CP)